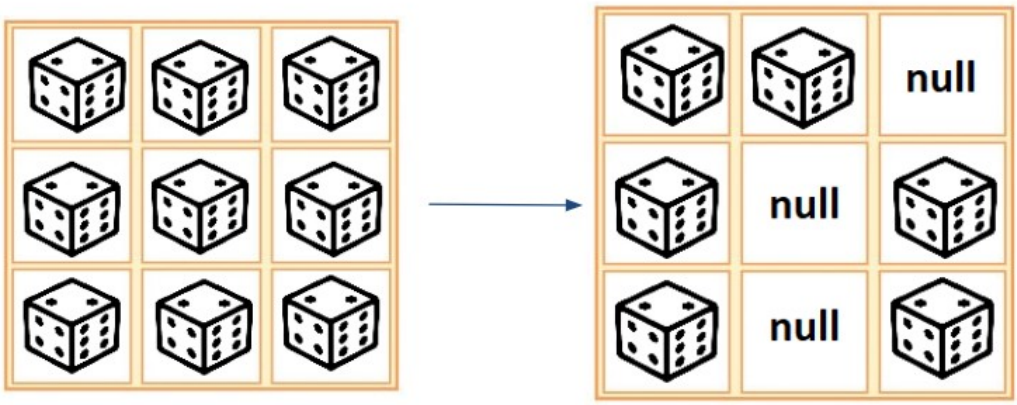
2a evaluación ENTREGABLE 1

Ejercicio A (*Ejercicio del dado*)

Objetivo: Crear una matriz de nxn dados que vayan lanzándose simultáneamente y todos los dados saquen en algún momento un 6

Requisitos:

- Crear la clase dado
- Crear una matriz de n x n objetos dado
- En cada iteración , lanzar todos los dados
- Si un dado saca 6, lo ponemos a NULL
- Si un dado está a NULL nunca más será lanzado
- El programa acaba cuando todos los dados están a NULL
- En cada iteración, imprime la situación de la matriz de dados



código base de la clase dado:

```
public class dado {
   int valor;

   public dado(int……)){
      this.valor =.....  ;
   }
   public int lanzar (){
      Random ……
     ……..
      System.out.println("Has sacado un: " + ……);
      return …… ;
   }
   public int mostrarValor (){

      System.out.println("Valor es: " + valor);
      return valor;
   }
}
```

Introduce aquí tu código la clase dado:

```java
import java.util.Random;

    public class dado {
    int valor;

    public dado(int valor ){
        this.valor =  valor  ;
    }
    public int lanzar ( ){
        Random random = new Random ();
        this.valor = random.nextInt(6) + 1;

        System.out.println("Has sacado un: " + this.valor);
        return  this.valor ;
    }
    public int mostrarValor (){

        System.out.println("Valor es: " + valor);
        return valor;
    }
}
```

Introduce aquí el resto del código:

```
public class TiraDados {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
    int n = 3;
    Dado[][] matriz = new Dado[n][n];

    for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        matriz[i][j] = new Dado();
    }
    }

    int iteracion = 1;
    boolean quedanDados = true;

    while (quedanDados) {
        System.out.println(" iteración " + iteracion + " ---");
        quedanDados = false;

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
            if (matriz[i][j] != null) {
                int resultado = matriz[i][j].lanzar();
                if (resultado == 6) {
                    matriz[i][j] = null;
                } else {
                    quedanDados = true;
                }
            }
            }
        }

            System.out.println(x: "Estado :");
            imprimirMatriz(matriz, n);
            iteracion++;
        }
        System.out.println(x: "los dados han sacado un 6. Fin .");
    }

    public static void imprimirMatriz(Dado[][] matriz, int n) {    Exporting non-public type through public API
        for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
        if (matriz[i][j] == null) {
        System.out.print(s: "[ X ] ");
        } else {
        System.out.print("[ " + matriz[i][j].mostrarValor() + " ] ");
        }
        }
```

```
        System.out.println();
        }
    }
}
```

```
 iteración 1 ---
Has sacado un: 4
Has sacado un: 3
Has sacado un: 4
Has sacado un: 2
Has sacado un: 3
Has sacado un: 5
Has sacado un: 1
Has sacado un: 5
Has sacado un: 6
Estado :
[ 4 ] [ 3 ] [ 4 ]
[ 2 ] [ 3 ] [ 5 ]
[ 1 ] [ 5 ] [ X ]
 iteración 2 ---
Has sacado un: 1
Has sacado un: 4
Has sacado un: 1
Has sacado un: 1
Has sacado un: 3
Has sacado un: 5
Has sacado un: 5
Has sacado un: 1
Estado :
[ 1 ] [ 4 ] [ 1 ]
[ 1 ] [ 3 ] [ 5 ]
```

Muestra un pantallazo de la parte final de la salida tras ejecutar tu código:

```
 iteración 7 ---
Has sacado un: 5
Has sacado un: 4
Has sacado un: 2
Has sacado un: 6
Estado :
[ 5 ] [ 4 ] [ X ]
[ X ] [ 2 ] [ X ]
[ X ] [ X ] [ X ]
 iteración 8 ---
Has sacado un: 2
Has sacado un: 1
Has sacado un: 6
Estado :
[ 2 ] [ 1 ] [ X ]
[ X ] [ X ] [ X ]
[ X ] [ X ] [ X ]
 iteración 9 ---
Has sacado un: 3
Has sacado un: 5
Estado :
[ 3 ] [ 5 ] [ X ]
[ X ] [ X ] [ X ]
[ X ] [ X ] [ X ]
 iteración 10 ---
```

```
 iteración 27 ---
Has sacado un: 1
Estado :
[ X ] [ 1 ] [ X ]
[ X ] [ X ] [ X ]
[ X ] [ X ] [ X ]
 iteración 28 ---
Has sacado un: 1
Estado :
[ X ] [ 1 ] [ X ]
[ X ] [ X ] [ X ]
[ X ] [ X ] [ X ]
 iteración 29 ---
Has sacado un: 2
Estado :
[ X ] [ 2 ] [ X ]
[ X ] [ X ] [ X ]
[ X ] [ X ] [ X ]
 iteración 30 ---
Has sacado un: 6
Estado :
[ X ] [ X ] [ X ]
[ X ] [ X ] [ X ]
[ X ] [ X ] [ X ]
los dados han sacado un 6. Fin .
```

Ejercicio B (subclase + toString)

Objetivo:
Sobreescribir el método toString y crear una subclase propia inventada por el alumno derivada de animal->mamífero (o ave o anfibio) y sobreescribir el método toString

Requisitos:

- Reusa una de las 3 subclases que propusistes en el ejercicio de animales

- SobreEscribe el método toString para que en vez de imprimir la referencia, imprima todos los valores de los atributos del objeto incluyendo los atributos heredados-

Introduce aquí tu código de la subclase elegida:

```java
public static class Orangutan extends Mamifero {
    private String saluda;    Field saluda can be final

    public Orangutan(boolean criando, String saluda, String elnombre, int edad, S
        super(criando, elnombre, edad, sexo, peso, despierto);
        this.saluda = saluda;
    }

    public void hacerSaludo() {
        System.out.println("El orangután : " + saluda);
    }
```

Introduce aquí el método toString:

```java
        }

        @Override
        public String toString() {
            return "Orangután: " + saluda;
        }
    }
}
```

Muestra un pantallazo del resultado final tras ejecutar tu código:

3eansProjects > animales > src > main > java > com > mycompany > Animal3 > Animal3.java > Java > Animal3 > main(String[] args)

```java
3   public class Animal3 {
        Run main | Debug main | Run | Debug
5       public static void main(String[] args) {
6               Mamifero.Orangutan mono = new Mamifero.Orangutan(criando: false, saluda: "H
7
8           animalC animal1= new animalC (elnombre: "Dumbo" , edad: 25,sexo: "hembra",peso
9          animal1.showinfo();
10
11
12          animal1.despertarse();
13          animal1.comer();
14          animal1.comer();
15          animal1.perderPeso();
16          animal1.dormir();
17            System.out.println(x: "");
18          System.out.println(x: "Mono");
19            System.out.println(x: "");
20          mono.showinfo();
21          mono.hacerSaludo();
22          mono.comer();
23      }
24
```

PROBLEMAS 13 SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS Run: Animal3

```
Jorge  ha comido Peso actual: 141
PS C:\Users\Usuario> ^C
PS C:\Users\Usuario>
PS C:\Users\Usuario>  & 'C:\Program Files\Java\jdk-25\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExc
eptionMessages' '-cp' 'C:\Users\Usuario\AppData\Local\Temp\vscodesws_6bdb9\jdt_ws\jdt.ls-java-project\bin' 'com.m
ycompany.animales.Animal3'
Informacion:Nombre: Dumbo
edad: 25
sexo: hembra
peso: 300
despierto: true
Dumbo se ha despertado
Dumbo ha comido Peso actual: 301
Dumbo ha comido Peso actual: 302
Dumbo perdio peso Peso actual: 301
Dumbo esta durmiendo.

Mono

Informacion:Nombre: Jorge
edad: 34
sexo: Macho
peso: 140
despierto: true
El orangután : Hola
Jorge  ha comido Peso actual: 141
PS C:\Users\Usuario>
```

EMA
A DE TIEMPO
DDE POKÉMON
ECTS
CONFIGURATION
PROJECTS

Herramienta Recortes

Captura de pantalla copiada en el portapapeles
Guardado automáticamente en la carpeta de capturas de pantalla.

Marcado y uso compartido